

COL380 Assignment 4

PV Revanth (2022CS51650)

1 Introduction

This report presents the parallelization strategy and performance analysis of a distributed matrix multiplication system designed for large sparse matrices. The implementation combines three parallel computing paradigms:

- **MPI (Message Passing Interface)** for distributed communication between compute nodes
- **OpenMP** for shared-memory parallelism within individual nodes
- **CUDA** for GPU-accelerated computation of matrix blocks

The system measures end-to-end execution times including file I/O operations, computational processing, and inter-node communication.

2 Parallelization Strategy

2.1 Parallel Matrix Input Handling

The implementation optimizes matrix reading through distributed I/O processing:

- **Work Distribution:** Matrices are divided equally among MPI processes
- **Thread Parallelism:** OpenMP threads within each process concurrently read assigned files
- **Race Condition are prevented:** Critical sections synchronize file access operations

2.2 Hybrid CPU-GPU Computation

Each MPI process employs a dual-strategy approach for matrix multiplication:

- **GPU Acceleration:**
 - `multiply_pair_cuda` handles initial matrix products

- Batched CUDA kernel (`batchedCudaMatMul`) processes block pairs in parallel
- Edge block dimensions dynamically calculated: $\min(k, \text{remaining_dimension})$

- **CPU Multithreading:**

- `do_internal_mul` processes subsequent multiplications
- OpenMP tasks identify compatible block pairs using map structures
- Atomic operations ensure thread-safe result accumulation

Key Optimizations:

- Computational load balancing between GPU and CPU
- Concurrent execution of matrix operations
- Batched memory transfers for GPU efficiency

2.3 Distributed Result Reduction

The system employs logarithmic-scale result aggregation:

- **Binary Tree Protocol:** Processes are paired using XOR-based partner selection
- **Communication Phases:** $\log_2(P)$ steps for P processes
- **Result Merging:** Receivers combine matrices using `do_internal_mul`
- **Progressive Deactivation:** Senders become inactive after transmission

2.4 Output Generation

The root process (Rank 0) writes the final matrix:

- **Block-wise Format:** Stores metadata (row, column, block size) with each block
- **Edge Handling:** Zero-padding for partial blocks at matrix boundaries

2.5 Final Output

3 Performance Analysis

Table 1: Performance Summary (16 Threads)

Nodes	Max Read Time (Node)	Max Mult. Time (Node)	Reduction Time	Total Time	IPC
1	0.104 s (0)	6.996 s (0)	1.31×10^{-5} s	7.368 s	1.35
2	0.084 s (1)	3.373 s (1)	0.100 s	3.856 s	1.18
4	0.157 s (2)	4.024 s (3)	0.233 s	4.731 s	1.15
8	0.048 s (1)	3.545 s (0)	0.681 s	4.582 s	1.39

Table 2: Performance Summary (4 Threads)

Nodes	Max Read Time (Node)	Max Mult. Time (Node)	Reduction Time	Total Time	IPC
1	0.323 s (0)	11.491 s (0)	6.74×10^{-6} s	12.037 s	1.91
2	0.506 s (1)	6.770 s (0)	0.289 s	7.823 s	1.88
4	0.245 s (1)	4.628 s (0)	0.495 s	5.488 s	1.70
8	0.089 s (1)	6.504 s (1)	1.909 s	8.768 s	1.76